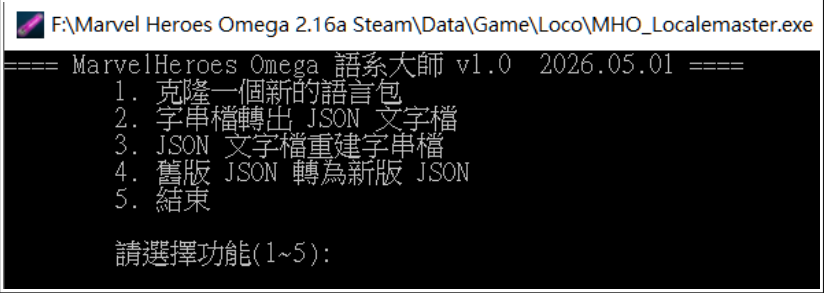
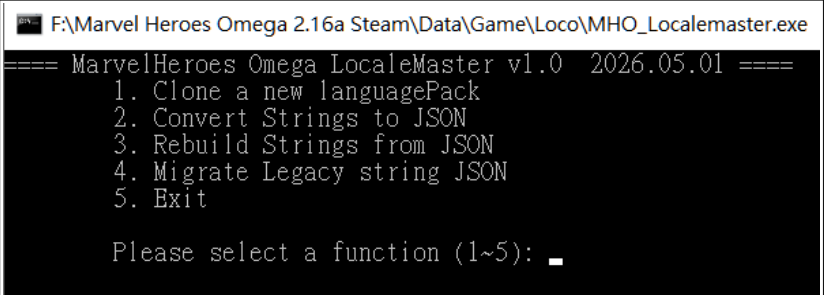


● 關於本軟體：

- 可以完整解決 MHO 遊戲的語言環境問題, 包括：新增語言, 將遊戲字串檔轉成文字檔, 將文字檔重建為字串檔。
開發本軟體的最主要目的是為了中文化 Marvel Heroes Omega, 在設計上是以翻譯整個語言字串檔為主要考量。
(上述的"新增語言"只是克隆遊戲內建的 6 國語言, 需再自行翻譯其內容。)
- 使用 Python 開發, ChatGPT 輔助完成。
 - 歡迎無償散佈/使用, 本軟體並未經廣泛測試, 不保證相容性與可靠度。
 - 本程式僅於 Windows 10 x64 中文版測試 - 完成了遊戲的中文版。
 - 所有英文訊息內容以 ChatGPT+ Gemini 翻譯完成。
 - 本程式設計用於遊戲 1.52 版, 未測試與其它版本的相容性。
 - 遊戲繁中版試作展示影片: <https://youtu.be/x3FhU4aS6Zg>
- MHO 語言環境資訊來自 MHServerEMU 專案, 感謝專案成員們。
<https://github.com/Crypto137/MHServerEmu/blob/master/docs/GameData/Locale.md>
<https://discord.com/invite/hjR8Bj52t3>

目錄： . 關於 MHO 語言環境的基本知識 0. 開始 1. 克隆一個新的語言包 2. 字串檔轉出 Json 文字檔 3. Json 文字檔重建字串檔 4. 舊版 Json 轉為新版 Json 5. 關於詞元: \$, # 和 #/ 6. 關於程式產生的檔案們 7. 翻譯一個語言包的流程	<pre>F:\Marvel Heroes Omega 2.16a Steam\Data\Game\Loco\MHO_Localemaster.exe ==== MarvelHeroes Omega 語系大師 v1.0 2026.05.01 ==== 1. 克隆一個新的語言包 2. 字串檔轉出 JSON 文字檔 3. JSON 文字檔重建字串檔 4. 舊版 JSON 轉為新版 JSON 5. 結束 請選擇功能(1~5):</pre> <pre>F:\Marvel Heroes Omega 2.16a Steam\Data\Game\Loco\MHO_Localemaster.exe ==== MarvelHeroes Omega LocaleMaster v1.0 2026.05.01 ==== 1. Clone a new languagePack 2. Convert Strings to JSON 3. Rebuild Strings from JSON 4. Migrate Legacy string JSON 5. Exit Please select a function (1~5): _</pre>
---	---

● . 關於 MHO 語言環境的基本知識

MHO 內建 6 種語言：英/法/德/葡萄牙/俄/西班牙，和 3 個已規劃未實作的語言：中/日/韓，以及 3 個不明用途的語言 (sg1/sg2/sg3)，合計 12 語言，本程式只能運作前 9 種語言。

MHO 的語言包，每組包含 1 個語言設定檔，1 個字串檔資料夾，和 4 個字串檔案。

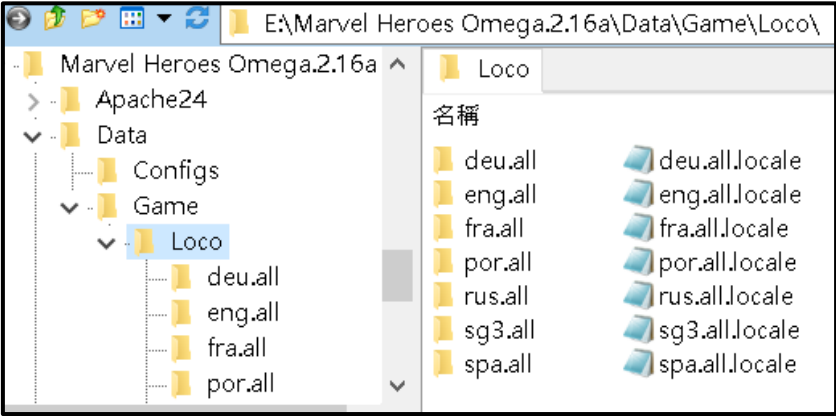
語言包的路徑如下圖，位於 \遊戲資料夾\Data\Game\Loco\：

如右圖，.\Marvel Heroes Omega.2.16a\就是{遊戲資料夾}。

遊戲資料夾名稱可自行修改，每個人可能不同，所以用{遊戲資料夾}稱呼。

eng.all.locale 是英文語言包設定檔。
eng.all 是英文語言包的字串檔資料夾。

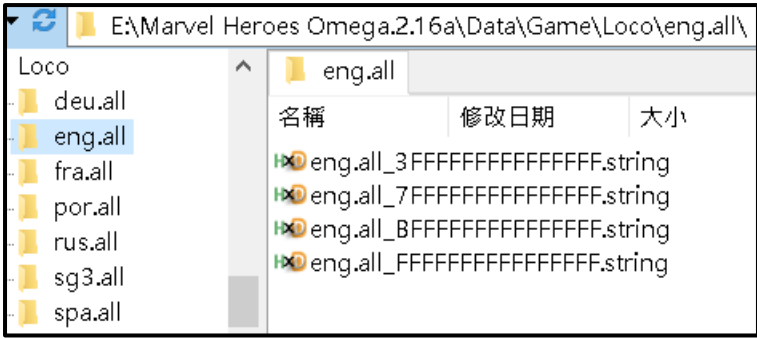
請留意，英文語言包的檔名是"eng.all"，而不是 "eng"，其它語言包亦同。



右圖是資料夾 \eng.all\ 的內容，有 4 個字串檔，檔名為 eng.all_*.string。

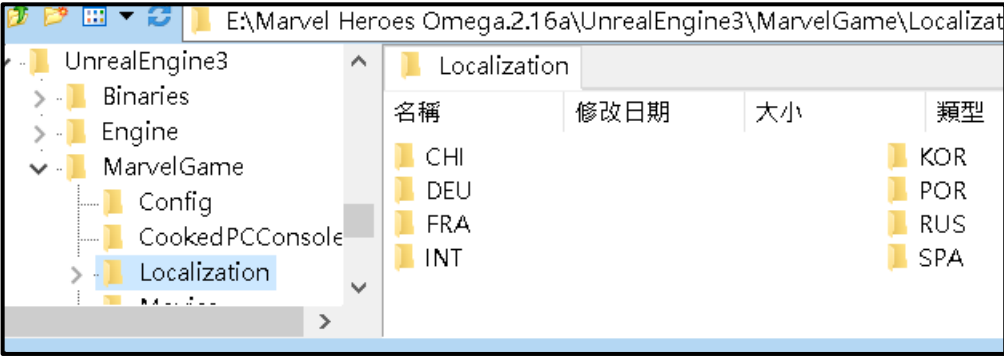
字串檔是 MHO 遊戲訊息檔，遊戲大部份的文字內容都在此。

其它不在此列的文字多半是由伺服器端提供，例如：成就，排行榜...



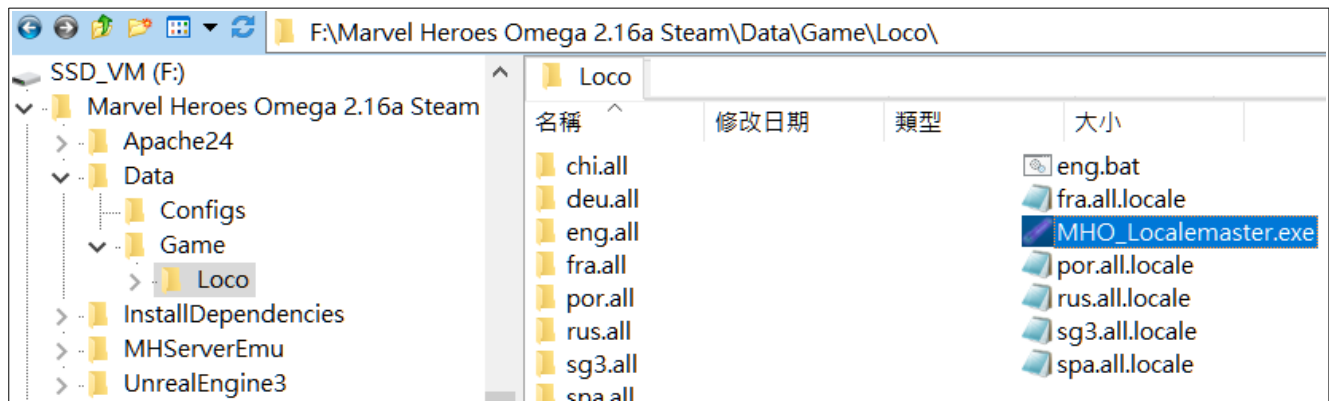
除了語言包，遊戲過場動畫也有多國字幕，已包含 中文/韓文字幕 (無日文字幕)。之前因為沒有 中/韓 語言包可啟用語言，所以有字幕也不能看，現在可以嘍。

下圖為字幕檔路徑：.\遊戲資料夾\UnrealEngine3\MarvelGame\Localization\



● 0. 開始

- 本工具只有 [MHO Localemaster.exe](#) 一個檔案, 使用時要一定要放在 `\遊戲資料夾\Data\Game\Loco\` 中, 如下圖:
(必須是完整遊戲資料夾的 `\Loco\`, 不能只是 copy 出一份 `\Loco\` 資料夾, 會檢查遊戲執行檔的版本...)



- 程式支援 繁中/英 雙語介面, 依 Windows 語言版本自動選擇, 或以參數指定, "[MHO_Localemaster.exe 參數](#)", 0: 英文, 1: 中文.
- 使用此工具之前, 宜先關閉遊戲.
- 在讀取字串 Json 檔時, 會自動檢測該 Json 檔是否格式正常, 並列出異常處, 每次只能列出發現的第一處異常.
- 本程式在進行破壞性工作之前, 都會自動備份.

- 本程式有兩個應用方向:

1. 修改現有語言:

可於英/法/德/葡萄牙/俄/西班牙/中/日/韓 9 種語言, 將目標語言生成 Json 文字檔, 編輯文字後, 再重建為新字串檔.

2. 產生新語言:

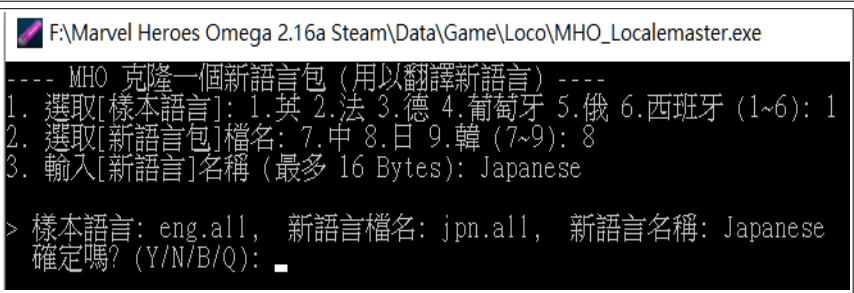
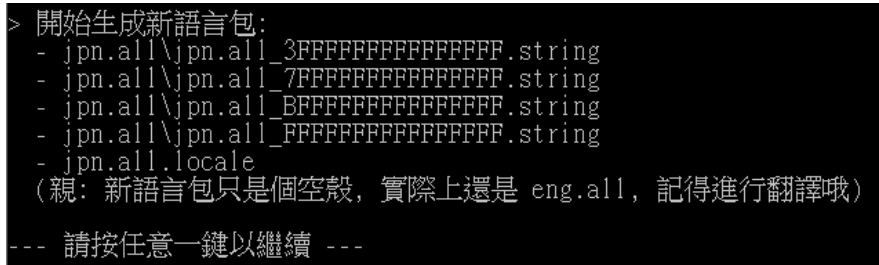
新語言狹義是指 中/日/韓 語, 廣意可以是任何語言... 只要 `MarvelFonts_SF.upk` 可以顯示目標語言的文字就行.

建立 .Locale, 克隆語言包資料夾和語言字串檔, 為新語言生成 Json 文字檔, 翻譯並重建字串檔.

● 1. 克隆一個新的語言包：

克隆一個現有語言, 使用 中/日/韓 任一個語言包(資料夾).
功能中的"新語言包檔名"固定是 chi.all, jpn.all 或 kor.all, 這只是三選一當作新語言包的資料夾.
然而**不管是語言包檔名, 還是新語言名稱. 都與此語言包真正的"語言內容"無關**, 真正決定語言內容的是[功能 2]產生的 string Json 檔.

舉實例來說, 下面圖例建立了 japanese 這個新語言包, 使用 jpn.all 當語言包資料夾, 將後續[功能 2]產生的 string json 檔翻譯為中文, 再用[功能 3]重建為 jpn.all 字串檔, 最終在遊戲中, 選這個 japanese 語言包看見的會是中文訊息. (實測結果)

步驟： 1. 選擇樣本語言, 六選一. 2. 選擇新語言包檔名, 三選一. 3. 輸入新語言名稱, 這會顯示在遊戲的語言選單.	
結果： - 生成 5 個檔案, 1 個資料夾. - 如果檔案/資料夾已存在, 會先進行備份.	
遊戲範例： 9 種語言同時上線.	

● 2. 字串檔轉出 Json 文字檔

本功能是将目標語言的字串檔轉成文字檔，產生 5 個檔案：
MainMap.json, SrcSampTok.json, StringMap.json 是主要檔案, 只有 StringMap.json 可以編輯。
dbg-locked_tokens.json 和 dbg-src_unpaired_tags.json 是樣本語言的勘誤檔, 非必要, 僅供參考。
(各檔案的解說, 請參閱 ● 6.)

步驟： 1. 選擇樣本語言, 樣本必須是已存在的語言. 2. 選擇轉出語言, 轉出語言若不存在, 會直接建立語言包資料夾(不含 .locale 檔). - 除非[樣本語言]和[轉出語言]相同 (修改原生語言) , 否則不建議選內建六語言當轉出語言.	F:\Marvel Heroes Omega 2.16a Steam\Data\Game\Loco\MHO_Localemaster.exe ---- MHO 語系字串檔轉出 JSON (用以翻譯/修改字串內文) ---- 1. 選取[樣本語言]: 1.英 2.法 3.德 4.葡萄牙 5.俄 6.西班牙 7.中 8.日 9.韓 (1~9): 1 2. 選取[轉出語言]: 1.英 2.法 3.德 4.葡萄牙 5.俄 6.西班牙 7.中 8.日 9.韓 (1~9): 8 > 樣本語言: eng.all, 轉出語言: jpn.all 確定嗎? (Y/N/B/Q):
結果： - 確認後, 在[轉出語言]資料夾產生 5 個 Json 檔. - 本功能會自動備份語言包資料夾.	> 輸出資料夾: jpn.all, 生成檔案: - jpn.all.SrcSampTok.json (校正樣本, 不可修改) - jpn.all.dbg-locked_tokens.json (已鎖定字串清單, 僅供參考) - jpn.all.dbg-src_unpaired_tags.json (未配對文字標籤清單, 僅供參考) - jpn.all.MainMap.json (字串檔主資料表, 不可修改) - jpn.all.StringMap.json (字串檔文字內容, 可編輯字串文字) --- 請按任意一鍵以繼續 ---
範例： jpn.all.StringMap.json 遊戲訊息文字檔的內容. - 應當編輯 "string": "..." 黃底區的文字. - status 只能修改 lock 和 no_audit 的值 (0, 1). dollar/hash/hash_slash 僅供參考, 修改無用. - dollar/hash/hash_slash 是顯示此字串內有多少個 \$變數, #標籤, #/標籤. - lock =1, 代表此字串鎖定, 不會使用此 string 值, 而是保持樣本語言的字串. - no_audit =1, 代表不和樣本語言驗算 \$, #, #/ 的數量, 此字串不會產生 log. - creation_time 不可擅改, 這是和另外兩個 json 檔的驗證碼.	<pre>{ "creation_time": "2026-05-06 01:27:20", "entries": [{ "seq_no": 1, "status": "lock=0, no_audit=0, dollar=0, hash=0, hash_slash=0", "string": "" }, { "seq_no": 2, "status": "lock=0, no_audit=0, dollar=0, hash=0, hash_slash=0", "string": "Bulletproof. Figures...\n—Punisher" }, { "seq_no": 3, "status": "lock=0, no_audit=0, dollar=0, hash=0, hash_slash=0", "string": "Mission Failure" }, { "seq_no": 4, "status": "lock=0, no_audit=0, dollar=0, hash=0, hash_slash=0", "string": "HYDRA Excavation" }] }</pre>

● 3. Json 文字檔重建字串檔

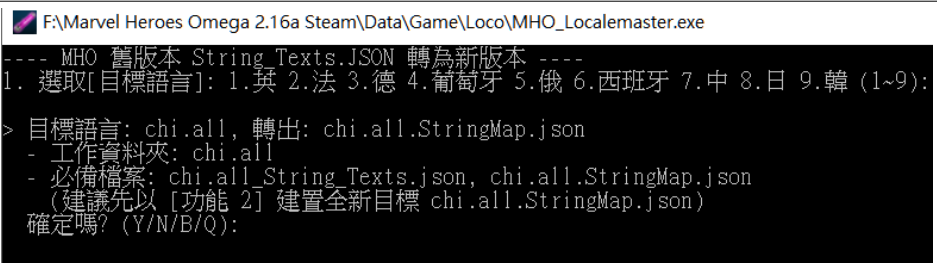
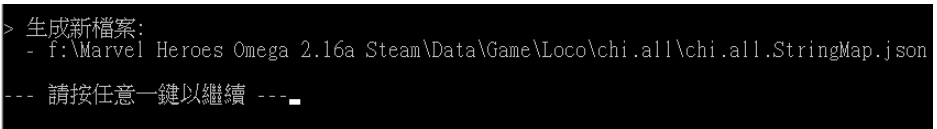
將 3 個主要 Json 檔重建為 4 個字串檔, 已存在的字串檔會直接覆寫, 不作備份。
目標語言資料夾裡必須有[功能 2]產生的同一組 MainMap.json, SrcSampTok.json, StringMap.json.

步驟： 1. 選擇目標語言 2. 確認無誤後, 直接生成目標語言字串檔.	F:\Marvel Heroes Omega 2.16a Steam\Data\Game\Loco\MHO_Localemaster.exe ---- MHO JSON 文字檔重建字串檔 (生成翻譯/修改的字串檔) ---- 1. 選取[目標語言]: 1.英 2.法 3.德 4.葡萄牙 5.俄 6.西班牙 7.中 8.日 9.韓 (1~9): 8 > 目標語言: jpn.all, 重建: jpn.all_{3/7/B/F}FFFFFFFFFFFFFFFF.string - 工作資料夾: jpn.all - 必備檔案: jpn.all.MainMap.json, jpn.all.StringMap.json, jpn.all.SrcSampTok.json 確定嗎? (Y/N/B/Q):
結果： - 除了四個字串檔外, 還會多一個 .Log 檔. Log 檔會記錄 \$, #, #/ 詞元數量不符的字串. 每次使用[功能 3] 都會覆寫 .Log 檔.	> 檢測 JSON 檔一致性: - jpn.all.MainMap.json 建檔時間: 2026-05-06 01:27:20 - jpn.all.StringMap.json 建檔時間: 2026-05-06 01:27:20 - jpn.all.SrcSampTok.json 建檔時間: 2026-05-06 01:27:20 > 生成新字串檔: - jpn.all\jpn.all_3FFFFFFFFFFFFFFFFF.string - jpn.all\jpn.all_7FFFFFFFFFFFFFFFFF.string - jpn.all\jpn.all_BFFFFFFFFFFFFFFFFF.string - jpn.all\jpn.all_FFFFFFFFFFFFFFFFFF.string - jpn.all.dbg-audit_tokens.log (字串的[\$變數,#標籤]校對記錄) ---- 請按任意一鍵以繼續 ----
範例： 右圖是 Log 檔的內容. 會列出有問題的字串流水序號, [原始字串]內容與[目標字串]內容供比對查對. 異常詞元即數量不符的 \$, #, #/ 例如: #mission#: 9/8: 表示#mission# 在原始字串有 8 個, 在目標字串卻有 9 個, 是誤改, 還是故意修正? 如果是後者, 可以設定 no_audit =1, 不要檢查這筆字串.	本檔案列出 StringMap.json 中[\$ 變數]和[# , #/ 標籤]與原始記錄不符之筆數, 除錯用 ----- 流水序號= "seq_no":1086 原始字串= Use any waypoint or terminal to travel to #mission#Midtown#/mission or 目標字串= 使用任何路標或終端機傳送到#mission#中城#/mission#或是#mission#神盾局全 異常符標= #mission#: 9/8, #/mission#: 9/8 ----- 流水序號= "seq_no":1347 原始字串= #waypointheader#\$RegionName\$#/waypointheader# #waypointlabel#LOCATI 目標字串= #waypointheader#\$RegionName\$#/waypointheader# #waypointlabel#地點:# 異常符標= #/waypointlabel#: 2/1 ----- 流水序號= "seq_no":1629 原始字串= #waypointheader#Airfield Showdown (Cosmic)#/waypointheader# #waypoi 目標字串= #waypointheader#機場決戰 (宇宙)#/waypointheader# #waypointlabel#地黑 異常符標= #/waypointlabel#: 2/1 -----

● 4. 舊版 Json 檔轉為新版 Json

本程式是重新改寫的, 之前曾有舊版本, 此功能可把舊 Json 檔轉為新 Json 檔.

- 前置條件:
- 舊格式檔: ???_all_String_Terms.json, ???_all_String_Texts.json.
 - [樣本/目標]語言都與舊檔相同的新版 Json 檔一組.
 - 將兩個舊檔和三個新檔都放進目標語言包資料夾中.

步驟: 1. 選擇目標語言 2. 確認無誤後, 生成新版 json 檔.	
結果: 本功能其實是把舊版檔案內的字串, 搬到新版檔案中, 所以才需要預先備妥一組新版檔案.	

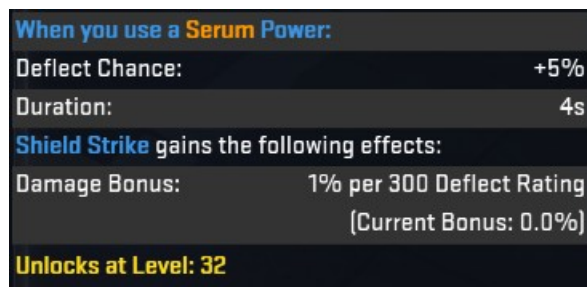
● 5. 關於詞元 (Token) : \$ dollar , # hash 和 #/ hash_slash

以右圖為例, "Unlocks at Level: 32" 在字串檔中實際是:

"Unlocks at Level: \$unlocklevel\$"; 32 是一個變數值, 變數由兩個\$號包夾, 變數不可以更動(翻譯).

要特別注意的是\$號中間的變數名稱- unlocklevel, 通常會對映一個相同的獨立字串 "unlocklevel", 此字串也不能更動.

\$變數名稱\$, 在 log 檔記錄為一個 dollar (\$).



再來看 "When you use a Serum Power:", 在字串檔中實際是:

"#emphasis#When you use a #powerkeyword#\$procKeyword.param2.keyword\$#/powerkeyword# Power:#/emphasis#"

Serum 是變數 \$procKeyword.param2.keyword\$ 的值.

#powerkeyword# 和 #/powerkeyword# 所包夾的 Serum 會變成粗體橘字, #格式標籤-開始# 和 #/格式標籤-結束# 會改變所包夾文字的格式, 標籤-開始是由 # #包夾標籤, 標籤-結束 是由#/ #包夾標籤, 正常來說, #標籤# 和 #/標籤#必須是一對, 如果不成對就是 BUG.

和\$變數\$一樣, #格式標籤# 和 #/格式標籤# 也是不可以更動(翻譯),

#標籤名稱#多半也有個相同的獨立字串存在: "標籤名稱", 同樣不能更動.

以上面兩個字串為例, 除了字串本身的 \$unlocklevel\$ 和 #emphasis#, #/emphasis#, #powerkeyword#, #/powerkeyword#, \$procKeyword.param2.keyword\$ 不該更動外, 如果另外找到以下 Entry 字串, 同樣不能更動:

"unlocklevel", "emphasis", "powerkeyword", "procKeyword.param2.keyword".

右圖是遊戲中, #標籤 異常最明顯的例子:

原英文字串如下:

"Use to claim a mystery reward! \n\n#QualityEpic# Epics: #/QualityEpic#\n\n*Gambit End of the World Costume (Exclusive)\n*Cyclops Noir Costume (Exclusive)\n*Bamf Pet\n*Ultimate Upgrade Token\n\n#RarityRare# Rares: #/RarityRare#\n\n*X-Force Red Spotlight Visual Artifact (Exclusive) \n*X-Men Blue Spotlight Visual Artifact (Exclusive) \n*Combo Boost\n\n*Infinity Orb Chest\n\n#rarityuncommon# Commons: #/rarityuncommon#\n\n*S.H.I.E.L.D. Supply Boost\n*Experience Boost\n*Currency Boost"

錯在哪裡? 標籤名稱獨立字串 "rarityuncommon" 不存在, 只找到 "RarityUnCommon".
標籤名稱改成 "RarityUnCommon" 後就好了, 中文版已修正.



#標籤名稱-開始#, 在 log 檔記錄為一個 hash (#), #/標籤名稱-結束#則是 hash_slash (#/)

● 6. 關於程式產生的檔案們

- 三個最重要的字串 Json 檔：

MainMap.json：記錄四個字串檔的主要數據資料，此檔案不該更改。

StringMap.json：只記錄字串檔的文字部份，讓翻譯人員能更專注於字串文字，略過 MainMap.json 裡的數據。

SrcSampTok.json：記錄樣本語言的字串文字，與各字串包含的 \$ 變數，# 標籤，#/ 標籤 數量。

- StringMap.json 是生成的所有檔案中，唯一可編輯的檔案，其它檔案都是不該修改的數據檔。

- StringMap.json 的每一筆字串都有 "status": "lock=0, no_audit=0, dollar=0, hash=0, hash_slash=0"，其中 lock 和 no_audit 是可以編輯的參數，dollar, hash, hash_slash 三者則僅是參考用數據，重建時不會回讀。

lock：預設值 0，若為 1 代表此字串被鎖定，不接受修改，強制使用 SrcSampTok.json 裡的字串原始值。

no_audit：預設值 0，若為 1 代表此字串不檢查 \$ 變數和 # 標籤的數量，也不會記錄。

- 兩個 Debug 用參考檔：

dbg-locked_tokens.json：

這個檔案列出所有的變數名稱和標籤名稱供參考，本程式預設會把所有和這些"名稱"相同的字串設定為 lock = 1，以避免誤改。當然，一定會有不是變數/標籤名稱的相同字串被誤鎖，這就要自行排除，或是直接將 lock=1 全部替換為 lock=0，解除這個預設功能。

以右圖為例，"Damage"和"Level"字串都有兩個，哪個是詞元名稱，哪個是真的遊戲文字呢？得自行排除嘍。

(右方的遊戲截圖文字看是全英文大寫，實際上是 "Level" 和 "Damage"，遊戲中有很多這樣的文字陷阱，翻譯過就知道：)

```
{
  "_comment": "本檔案列出預設 locked 的 str",
  "creation_time": "2026-05-07 14:35:01",
  "total": 658,
  "tokens": [
    "AreaName",
    "CurrentValue",
    "DISPLAYNAME",
    "Damage",
    "DamageOverTimeDuration",
    "LegendaryMissionsComplete",
    "Level",
    "LevelRange",
```

LEVEL	1
DAMAGE REDUCTION	11%
DAMAGE	0

dbg-src_unpaired_tags.json:

這個檔案列出樣本語言中，# 標籤#和#/ 標籤#不成對的字串，也就是原廠的標籤 BUG，應當優先修正。

以右圖的 1347 字串為例，有 2 個 #waypointlabel#，和 1 個 #/waypointlabel#，欠修理啦～

```
{
  "_comment": "本檔案列出樣本字串檔中未配對的文字標籤 (",
  "creation_time": "2026-05-07 14:35:01",
  "total": 93,
  "entries": [
    {
      "seq_no": 1347,
      "string": "#waypointheader#$RegionName$#/waypoint",
      "hash": { "#waypointlabel#": 2 },
      "hash_slash": { "#/waypointlabel#": 1 }
    },
    {
      "seq_no": 1629,
      "string": "#waypointheader#Airfield Showdown (Cos",
      "hash": { "#waypointlabel#": 2 },
      "hash_slash": { "#/waypointlabel#": 1 }
    },
    {
      "seq_no": 1968,
      "string": "I have nothing else for you right now.",
      "hash": { "#mission#": 2 }
    }
  ]
}
```

- 重建字串檔時產生的勘誤 log 檔：

dbg-audit_tokens.log：

這個檔案會記錄 StringMap.json 和樣本語言數量不符的詞元，Log 詞元不符不見得是壞事，當樣本字串是原廠 BUG，修正後就會 Log，若確認為目標字串是好的，可以設 no_audit=1，不要再檢查此筆字串。

● 7. 翻譯一個語言包的流程

步驟一:

使用 [功能 1] 克隆一個新的語言包, 例如: 想建立"中文", 語言名稱建議輸入"Chinese", 也可以輸入"中文", 中文使用 Utf-8 編碼, 可以輸入 4~5 個中文字. 樣本語言最佳選擇是英文, 新語言包檔名可於 Chi/Jpn/Kor 3 者任選, 只要是空置的就行. (有興趣的話, 可嘗試在 中日韓三個資料夾建立 繁中/簡中/粵語 試試:)

完成此功能後, 就能在遊戲中的語言選項看到新增的"Chinese", 選它後重新進入遊戲, 就會看到... 樣本語言版的遊戲. (其實還是有差別, 中文環境的"英文版"遊戲字體比較大)

步驟二:

使用 [功能 2] 字串檔轉出 Json 文字檔, 將樣本語言的字串檔轉出為 Json 文字檔, 翻譯 StringMap.json 就可以創造新語言... 或是編修現有的語言, 原理是一樣的. 若要修改現有的繁體中文當然也沒問題, 缺點就是要繼承大量的魔改和半英文字串 (笑).

步驟三:

開啟[功能 2]產生的 StringMap.json, 編輯裡面 "string": "文字串", "status" 列僅可修改 lock 和 no_audit, 值為 0/1 (布林). 翻譯 StringMap.json 會有很多坑, 只能累積經驗值硬磨, 最重要的就是前面第 5 章的詞元, 能處理好詞元, 問題就不大. 切記只有 StringMap.json 可以編輯, MainMap.json 和 SrcSampTok.json 不該有任何異動.

步驟四:

使用[功能 3] 將翻譯好的 StringMap.json 檔案群回寫為字串檔. 重複步驟 三, 四, 直到滿意為止.

工具:

- Notepad++ 是編輯器上選, 對中文的支援度最高.
- Q-Dir 之類的多窗式檔案總管, 方便在多個資料夾切換作業.